Alternative Coding Methods for Spiking LCA

Alex Zhang

October 2026

1 Latency coding

1.1 Definitions

Let $u_i^{(k)}$ be the state of the integrator at cycle k.
Let v be the state of the IF neuron attached to the integrator.
Let t_r be the received spike time of the recurrent weights.
Let t_f be the received spike time of the feedforward weights.
Let η be the LCA learning rate.
Let w_r be the recurrent weights leading to neuron i, with $w_r = \epsilon(I - \eta\Phi^T\Phi)$.
Let w_f be the feedforward weights leading to neuron i, with $w_f = \epsilon\eta\Phi^T$.
Let T be the cycle length, with time $0 \leq t < T$.
Let θ be the threshold for the attached IF neuron.
Let a be the coefficients of sparse coding.
Let x be the image vector.
Let λ be the threshold/sparsity penalty coefficient.
Let ϵ be the decay constant of $u_i^{(k)}$.

1.2 Derivations

The LCA formulation for the state $u^{(k+1)}$ is the following:

$$u^{(k+1)} = (1-\epsilon)u^{(k)} + w_r a^{(k)} + w_f x$$

So, for each $u_i^{(k+1)}$,

$$u_i^{(k+1)} = (1-\epsilon)u_i^{(k)} + \sum_j x_j w_{f_j} + \sum_j a_j w_{r_j}$$

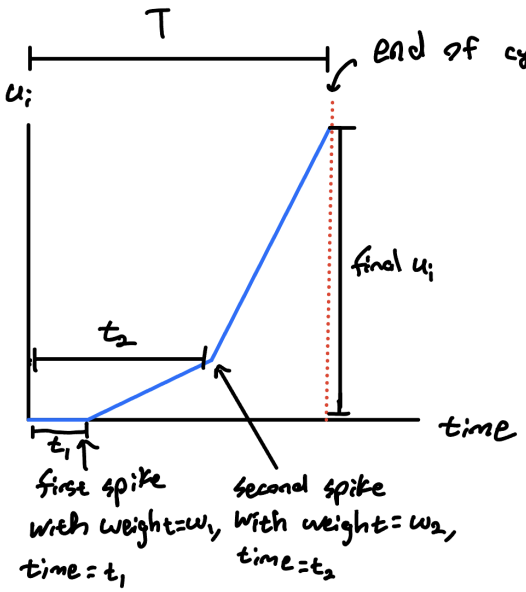
To begin, we must encode a_j and x_j:

$$T - t_{r_j} = a_j$$
$$T - t_{f_j} = x_j$$

It then follows that the state of the next cycle $u_i^{(k+1)}$ is given by:

$$u_i^{(k+1)} = (1-\epsilon)u_i^{(k)} + \sum_j (T - t_{f_j}^{(k+1)}) w_{f_j} + \sum_j (T - t_{r_j}^{(k+1)}) w_{r_j}$$

This equation describes an integrator where weights are summed at spike times. The following example illustrates this regime:

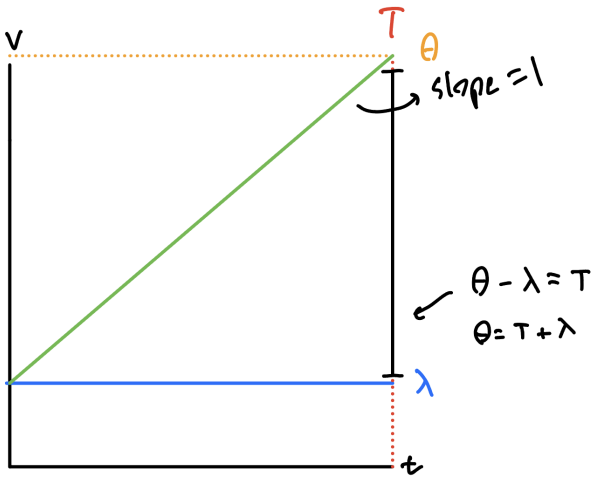


In this example, there are two spikes that come at times t_1 and t_2. The state u_i begins at the previous cycle's final u_i with a decay step applied (which for simplicity, we'll set to 0) and uses the two spikes' weights as input currents, meaning the slope from t_1 to t_2 is the weight w_1, and the slope from t_2 to the cycle end T is $w_1 + w_2$. From here, one can derive the final u_i in terms of the two spike times and the previous u_i to be $u_i = (t_2 - t_1)w_1 + (T - t_2)(w_2 + w_1) = (T - t_1)w_1 + (T - t_2)w_2$ (you can see how $(t_2 - t_1)w_1 + (T - t_2)(w_2 + w_1)$ sum to the height of the final u_i if you break the graph up into two triangles), which is equal to the aforementioned expression for $u_i^{(k+1)}$.

Furthermore, the next cycle's IF membrane state is determined by the following function on timestep t:

$$v_i^{(k+1)}(t) = u_i^{(k)} + t \text{ when } v_i < \theta \text{ and } 0 \text{ after } v_i \geq \theta$$

It has a constant input current of 1 and starts at $u_i^{(k)}$. The threshold θ is derived using the condition that the neuron should only fire if $u_i > \lambda$. The following diagram shows the derivation:



At the threshold, the constant input current should exactly reach the threshold at the end of the cycle, at time T. Thus, we can solve for θ in terms of T and λ by setting the distance $\theta - \lambda$ equal to the slope times distance T. As shown in the figure, $\theta = T + \lambda$.

From this, we can derive the output spike time $t^{(k+1)}$. The slope of 1 can be written as $1 = (\theta - u_i^{(k)})/t^{(k+1)} = (T + \lambda - u_i^{(k)})/t^{(k+1)}$, and solving for $t^{(k+1)}$, $t^{(k+1)} = (T + \lambda - u_i^{(k)})$. However, t cannot be negative, so we must enforce positivity: $t^{(k+1)} = \max(0, T - (u_i^{(k)} - \lambda))$. If there is a value $(u_i^{(k)} - \lambda) > T$, this value is too large for the representational range of the chosen cycle length T and input current of 1, and the earliest representable spike time (0) is applied. Of course, if $u_i + T < \theta = \lambda + T$, the neuron doesn't fire. If you plug back in the original $T - t_r = a_i$, you can see that, for the representable range, $a_i = u_i^{(k)} - \lambda$, matching LCA.

1.2.1 Corrections for Discrete time

In practice, discrete time is normally used, meaning we must account for the rounding error. Let S be the number of timesteps and dt be the time represented by each timestep, with $Sdt = T$. For example, if the true $u_i - \lambda$ is $4.5dt$, the true firing timestep is timestep $S - 4.5$, but it will only spike at timestep $S - 4.0$, as it only checks whether the threshold is breached at integer time steps. Thus, it fires 0.5 timesteps too late and represents a value of $4.0dt$, $0.5dt$ too low. We can correct for this rounding by carrying over the leftover v membrane potential to the next cycle. Over the next few cycles, this extra membrane potential contributes to a correction to the rounding. With $u_i - \lambda = 4.5dt$, carrying the leftover v over instead of overwriting would result in v having a membrane

potential of $0.5dt$ after the cycle finishes, carrying extra potential to the next cycle. After the next few cycles, the $0.5dt$ extra potential will cause some spikes to have slightly earlier firing times, correcting for the slightly late firing time of the original rounding error by adding that missing value to the state u_i in the next cycles. However, since u_i is a leaky integrator, this correction is not exact, as recent cycle leftovers are weighted higher than older ones.

Formally, the next cycle's IF membrane state would instead be defined:

$$v_i^{(k+1)}(t) = v_i^{(k)}(T) + u_i^{(k)} + tdt \text{ when } v_i < \theta \text{ and } v_i^{(k+1)}(t_{v_i \geq \theta}) - \theta \text{ after } v_i \geq \theta$$

for integers $t < S$.

2 Temporal Pattern Coding

2.1 Definitions

Let $u_i^{(k)}$ be the state of the integrator at cycle k.
Let v be the state of the IF neuron attached to the integrator.
Let T be the cycle length, in timesteps.
Let η be the LCA learning rate.
Let w_r be the recurrent weights leading to neuron i, with $w_r = \epsilon(I - \eta\Phi^T\Phi)$.
Let w_f be the feedforward weights leading to neuron i, with $w_f = \epsilon\eta\Phi^T$.
Let a be the coefficients of sparse coding.
Let x be the image vector.
Let λ be the threshold/sparsity penalty coefficient.
Let ϵ be the decay constant of $u_i^{(k)}$.
Let H be the Heaviside step function, with $H(t) = 1$ when $t \geq 0$ and $H(t) = 0$ otherwise.
Let θ be the threshold vector of length T, where the jth entry θ_j is defined as $\theta_j = 2^{T-j-1}$, so the zeroth entry would be 2^{T-1}, the first entry would be 2^{T-2}, etc.

2.2 Derivations

Here, we must break time into discrete timesteps (when implementing the previous, one must do the same), with each cycle having the defined T timesteps. This method utilizes all timesteps to their maximum extent, and with T timesteps, there are 2^T possible values to represent. We begin with encoding. Let $a_i = t_r$, where t_r is a binary vector of length T representing powers of 2. For example, if $t_r = [1, 0, 1, 0]$, then $\lfloor a_i \rfloor = 2^3 + 2^1 = 10$. Similarly, let $x_i = t_f$. Of course, we can scale x and a so that the maximum value we need to represent is near 2^T, meaning the granularity of representation is dependent on the cycle length T.

Thus, with this encoding, the state u_i as a function of timesteps t is defined as such:

$$u_i(0 \leq t < T) = \sum_p \sum_{t_{r_{j_p}}=1} w_{r_p} H(t - t_{r_{j_p}}) 2^{T-j-1} + \sum_p \sum_{t_{f_{j_p}}=1} w_{f_p} H(t - t_{f_{j_p}}) 2^{T-j-1}$$

where p denotes the index of a presynaptic neuron, and j denotes the time of spike $t_{f_{j_p}}$. Here, we set a globally shared gain of 2^{T-j-1} applied periodically every cycle.

When $t = T$, we apply a decay step $(1 - \epsilon)u_i$, meaning the next final state $u_i^{(k+1)}$ in terms of previous final state $u_i^{(k)}$ is defined as such:

$$u_i^{(k+1)} = (1 - \epsilon)u_i^{(k)} + \sum_p \sum_{t_{r_{j_p}}=1} w_{r_p} 2^{T-j-1} + \sum_p \sum_{t_{f_{j_p}}=1} w_{f_p} 2^{T-j-1}$$

as $H(T - t_{r_{j_p}}) = 1$ for all $t_{r_{j_p}}$. To decode the state u_i back into spikes along with applying thresholding, we construct an IF neuron with a threshold that halves every timestep and give the IF neuron a starting membrane potential of $u_i - \lambda$. Thus, the membrane potential v at each timestep t can be defined recursively:

$$v(t < T) \;=\; v(t-1) - \left(2^{(T-t-1)} \cdot H(v(t-1) - 2^{(T-t-1)})\right)$$

with base case $v^{(k)}(-1) = v^{(k-1)}(T) + u_i - \lambda$, as before timestep 0, v's initial value is $u_i - \lambda$ added to the previous final v value.[1] At each step t, $2^{(T-t-1)}$ is subtracted from the state $u_i - \lambda$ if $v \geq 2^{(T-t-1)}$. Explicitly, given that the starting $u_i - \lambda$ was less than 2^T, at any given t, $v(t) = u_i - \lambda - n(2^{(T-t-1)})$ for the maximum n that still keeps $v(t)$ nonnegative.[2] This subtracting also happens to be the nonnegative definition of a modulus, so an explicit formula of $v(t)$ is $v(t) = (u_i - \lambda) \mod 2^{(T-t-1)}$ when $0 \leq u_i - \lambda < 2^T$. Of course, when $u_i - \lambda < 0$, no subtracting is needed, and thus no firing happens in the cycle.

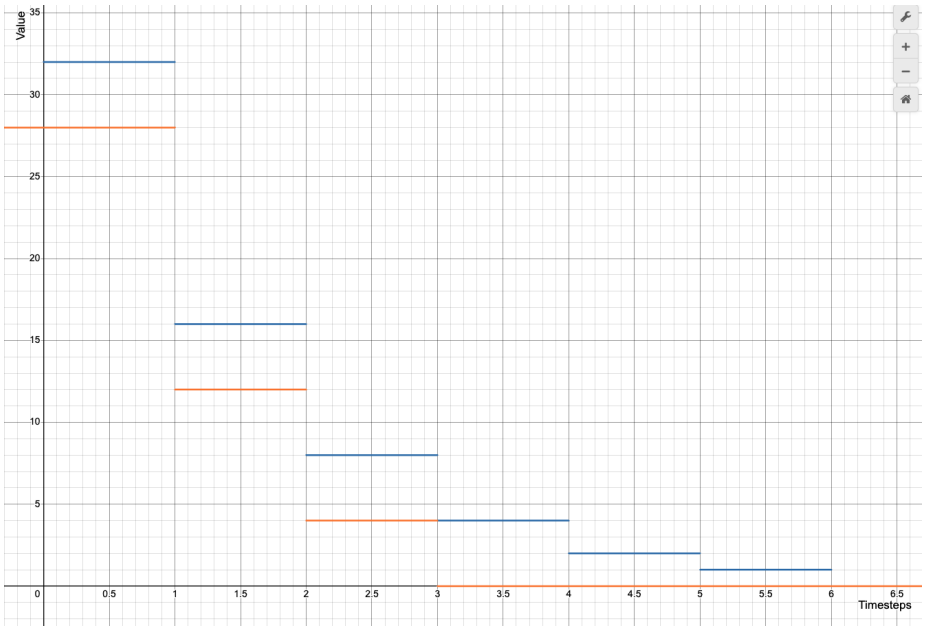
The output spike train t_r vector can be defined for representable values with a list comprehension:

$$t_r = [\lfloor \frac{\max(0, u_i - \lambda)}{2^{T-1-j}} \rfloor \mod 2 \text{ for } j \text{ in } [0, 1, 2, ..., T-1]]$$

[1] A similar rounding error correction applies to temporal coding LCA. If q is the smallest unit representable by the system (similar to the value of dt in latency coding), a value such as $8.2q$ would be represented as a spike train of $[..., 1, 0, 0, 0] = 8q$, meaning the represented is again slightly too low, requiring the carry correction.

[2] It can be shown that this recursion reaches the maximum n with a proof by contradiction and induction. First, we must establish that $0 \leq u_i - \lambda < 2^T$. Assume that for some t, the maximum n wasn't reached, meaning $v(t) \geq 2^{(T-t-1)}$. This means that in timestep $t - 1$, there must've been a spike fired and a subtractive reset applied, as $v(t)$ is already above the threshold. Thus, $v(t-1) \geq 2(2^{T-t-1}) = 2^{T-t}$. However, since 2^{T-t} is timestep $t - 1$'s threshold, the same argument applies here ($v(t-2) \geq 2(2^{T-t}) = 2^{T-t+1}$). By induction, $v(-1) = u_i - \lambda \geq 2^T$, contradicting the original assumption that $u_i - \lambda < 2^T$ and proving that the maximum n is reached for all timesteps t.

The $\lfloor \frac{\max(0, u_i - \lambda)}{2^{T-1-j}} \rfloor$ acts as integer division, and taking $\mod 2$ returns the binary digit at index j. An example u_i value is encoded into spikes in the diagram below.



The x-axis represents timesteps, and the y-axis represents value. The orange lines are the values of $u_i - \lambda$, and the blue lines are the values for θ_j. In this example, $T = 6$. The example value is $u_i - \lambda = 28$, and as timestep 0 passes, nothing happens to it, as $u_i - \lambda < \theta_0$ during timestep 0. At each following timestep, if $v \geq \theta_j$, θ_j gets subtracted from v and a spike is emitted. Thus, the timesteps where the spikes get emitted for $u_i - \lambda = 28$ are 1, 2, and 3, corresponding to $2^4 + 2^3 + 2^2 = 28$.

To confirm that this regime matches LCA up to its representational power, recall that the vectors we encoded a_i and x_i into are effectively the binary representations of the two respective values. Thus, summing the spikes with an exponential index weight, as $\sum_{t_{r_{j_p}}=1} w_{r_p} 2^{T-j-1}$ and $\sum_{t_{f_{j_p}}=1} w_{f_p} 2^{T-j-1}$ do in the expression for $u_i^{(k+1)}$, return back a_i and x_i, respectively, meaning the expression for $u_i^{(k+1)}$ is equivalent to the original LCA formulation $u_i^{(k+1)} = (1-\epsilon)u_i^{(k)} + \sum_j x_j w_{f_j} + \sum_j a_j w_{r_j}$. Further, as stated earlier, the IF neuron only spikes if $u_i - \lambda > 0$, matching the thresholding of LCA.

3 Graded Spikes (Loihi 2's capabilities)

3.1 Definitions

Let $u_i^{(k)}$ be the state of the integrator at cycle k.
Let η be the LCA learning rate.
Let w_r be the recurrent weights leading to neuron i, with $w_r = \epsilon(I - \eta\Phi^T\Phi)$.
Let w_f be the feedforward weights leading to neuron i, with $w_f = \epsilon\eta\Phi^T$.
Let a be the coefficients of sparse coding.
Let x be the image vector.
Let λ be the threshold/sparsity penalty coefficient.
Let ϵ be the decay constant of $u_i^{(k)}$.

3.2 Derivations

When we allow each spike to carry values beyond 1, mapping LCA onto spiking architecture becomes quite trivial. Classical LCA is defined as such:

$$u_i^{(k+1)} = (1-\epsilon)u_i^{(k)} + \sum_j x_j w_{f_j} + \sum_j a_j w_{r_j}$$

First, we normalize a and x so that they both contain values only between -1 and 1. We then define a network of thresholdless neurons with states u. All we need to do is allocate 1 timestep to each cycle k, and have each neuron transmit a spike with value $\text{round}(2^{n-1}\,\text{sign}(u_i)\max(0, |u_i| - \lambda))$ (or not spike; a spike with 0 value is just substituted with no spike), where n is the number of bits in the representation (32-bit for Loihi 2), as payload can only be integers. Then, we just rescale by $1/2^{n-1}$ at each receiving neuron accordingly. We don't even need to double inputs or double the dictionary, as graded spikes are signed!

4 Complex Sparse Coding and Phase coded SNNs